

DOMAIN NAME SYSTEM REDESIGN

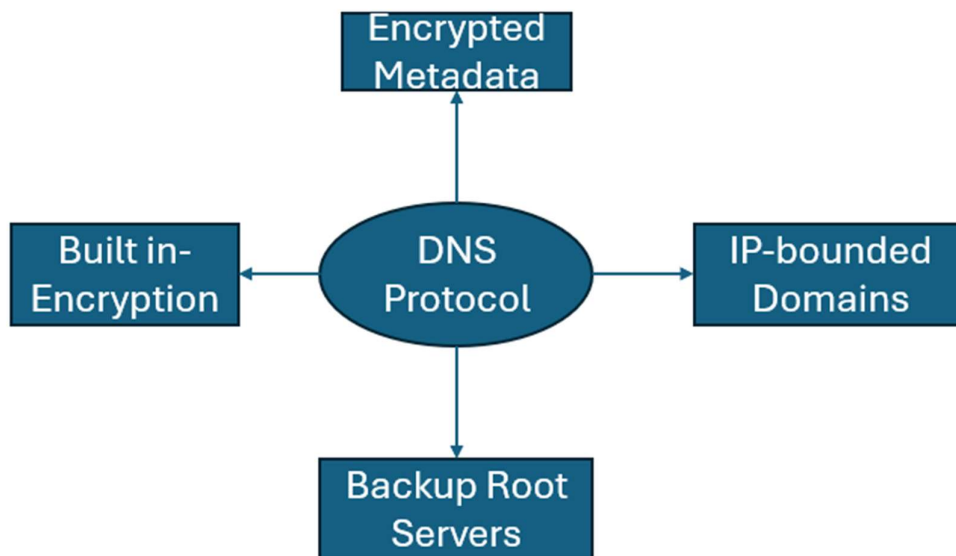
Contents

- Introduction 2**
- DNS Protocol Design 3**
- DNS Protocol Workflow 4**
- DNS Protocol Trade-offs..... 5**

Introduction

This redesign of the DNS protocol is designed to create a reliable, secure, private and resilient phonebook for the internet. This redesign was developed as currently DNS was made for a time where the internet could be trusted and wasn't as large and complex as it is now. This redesign aims to make DNS ready for the current world it is used in. This DNS redesign keeps the main functions of linking domains to IP addresses and using the DNS resolvers and root servers. This DNS will focus on encryption, verification and resilience instead of functionality.

This DNS redesign cannot promise perfect security, reliability, privacy or resilience as that is unachievable for any system. However, this DNS redesign can provide a more secure and resilient domain name system compared to the one we use today. This redesign is made for network protocol engineers. This is because these engineers have access to DNS protocol code and can make the changes detailed within this design. Make sure to adapt the concepts and details in this design as needed for implementation or use case.



The spider diagram shows the 4 main concepts used within this DNS protocol redesign. We can see built-in encryption which means that the protocol will be designed with encryption at the foundation. IP-bounded domains meaning IP addresses will be linked directly and verified against domains to protect against malicious sites. Backup root servers act as a resilient structure for if the main 13 root servers were to fail. Encrypted metadata protects the privacy of the users and against surveillance. You can find more detail about these concepts in the sections below.

DNS Protocol Design

- This design has 4 main changes from the current DNS protocol.
- The first change is built-in encryption.
- The DNS protocol will encrypt DNS queries by encrypting each packet individually using a pre-shared key or pre-query packet key.
- This means that during transit the queries are encrypted and are only decrypted once they reach the destination.
- This can be done using DH or other methods for the end-to-end encryption.
- At each stage of the hierarchy, only the domain is shown while only the rest of the data is decrypted once the authoritative name server has been reached.
- This is better over DOH as it doesn't rely on certificates which is third-party verification and presents a single point of failure.
- The second change is hop-to-hop verification for all servers
- Domains still managed by central authorities that handle handing out domains to companies which doesn't need to change.
- Each node within the DNS chain except for the client is signed with a private key created by the provider which could be 128 bits or more.
- Each node within the chain verifies the next using their public key to check the signature.
- The user's device is provided with a public key by the provider of the DNS resolver to verify the signature.
- If the signature breaks, the device will send out another query to try find the real DNS resolver.
- The third change is backup root servers.
- Currently, DNS uses 13 root servers which if they are taken down then the DNS protocol fails completely.
- This design uses an emergency protocol where DNS queries are sent to root undisclosed backup servers which can handle the DNS queries
- This emergency protocol is triggered when 6 or more root servers have experienced a critical failure to ensure that failures can be handled.
- These backup servers can be managed and maintained either by the central authorities or multiple different companies.
- Once the main servers are back up, the emergency protocol is turned off, and the main servers handle DNS queries once again.
- The fourth and final change is encrypted metadata.
- Currently, DNS metadata is leaked where an observer can see it and potentially use this data to compromise a device or system.
- This design wraps the metadata within the encrypted contents to ensure that an observer cannot read it.
- The header should only have the source IP address, destination IP address and domain for transportation.

DNS Protocol Workflow

- **This workflow will cover the process of a user making a query to ending with beginning of HTTP communication.**
- **First, the user opens their browser and enters “Tiktok.com” in the search bar then presses enter.**
- **Second, their device checks its local DNS cache to see if this domain has been used frequently.**
- **The device does not find the IP address of Tiktok.com in its local DNS cache so it moves up the hierarchy.**
- **Third, The device queries recursive resolver 1.1.1.1 for the IP address of TikTok.com.**
- **The device uses its public-key to verify the signature of the resolver which it doesn’t break.**
- **This resolver checks its cache for the IP address of TikTok.com but does not find it so sends the DNS query to a root server.**
- **The resolver checks the signature of the root server with its public key to ensure that it is talking to the correct root server.**
- **Fourth, the root server does not have the IP address of any domains but knows which TLD server to ask.**
- **The root server looks at the DNS query and sees .com and redirects the query to the TLD server that handles domains ending with .com**
- **The root server uses its public key to verify the signature of the TLD server.**
- **Fifth, the TLD server receives the query but doesn’t know the IP address of the domain. However, it does know which authoritative name server does.**
- **The TLD server sends the query to the authoritative name server so it can find the domain matching the IP address.**
- **The TLD server uses its public key to verify**
- **Sixth, the authoritative name server receives the query and checks its DNS records to find the IP address for Tiktok.com.**
- **The authoritative name server finds the IP address which is 23.216.154.118 for Tiktok.com.**
- **Seventh, the response is sent back through the hierarchy and cached at each level for future queries.**
- **This means that the response flows through authoritative name server → TLD server → root server → resolver → client.**
- **The response is encrypted and only the domain and IP address are seen by everything except the client and authoritative name server.**
- **Eighth, the response arrives back to the client, and the client immediately begins HTTP communication with the website for usage.**
- **Ninth, the client stores the IP address of Tiktok.com in its local DNS cache for faster lookups in the future.**

DNS Protocol Trade-offs

- This domain name system redesign has trade-offs just like any other system I have created.
- Developers and engineers may also face further deployment or implementation challenges during development.
- There are 7 trade-offs I have identified with this protocol.
- The first trade-off is time-consumption.
- This trade-off exists because it will take time to develop this protocol's encryption, verification and redundancy features.
- This trade-off can be managed by structured development processes, using existing DNS functions and clear allocation of responsibilities.
- The second trade-off is cost.
- This trade-off exists because it will be expensive to develop this protocol's resilience infrastructure especially with multiple servers.
- This trade-off can be managed by budget management, using existing infrastructure and efficient energy practices.
- The third trade-off is key management
- This trade-off exists because resolver providers will have to manage a private key to ensure that an attacker does not gain unauthorized access.
- This trade-off can be managed by storing the key in TPM or the password if used in a password manager to prevent exposure of the key.
- The fourth trade-off is complexity.
- This trade-off exists because engineers or developers may find it confusing to implement the features and systems used in this protocol.
- This trade-off can be managed by phased implementation, structured development team and testing the protocol in simulations to visualize.
- The fifth trade-off is migration.
- This trade-off exists because it will take time for companies and users to adopt the new updated protocol as it does with any new major redesign.
- This trade-off can be managed by phasing deployment starting with high-risk environments and updating the main DNS protocol instead of creating a completely new protocol.
- The sixth trade-off is compatibility.
- This trade-off exists because some DNS features in this redesign may not be supported by legacy systems.
- This trade-off can be managed by testing the protocol on these legacy systems before implementing it fully.
- The seventh trade-off is performance.
- This trade-off exists because due to the encryption and verification DNS responses may be slower than they are now.
- This trade-off is acceptable in terms of achieving a high level of security.